KU LEUVEN

Code Like a Researcher: Organizing Your (Python) Projects

For your (and your teachers) sanity

Alexandre Personnic

November 2025



Why Should You Care?



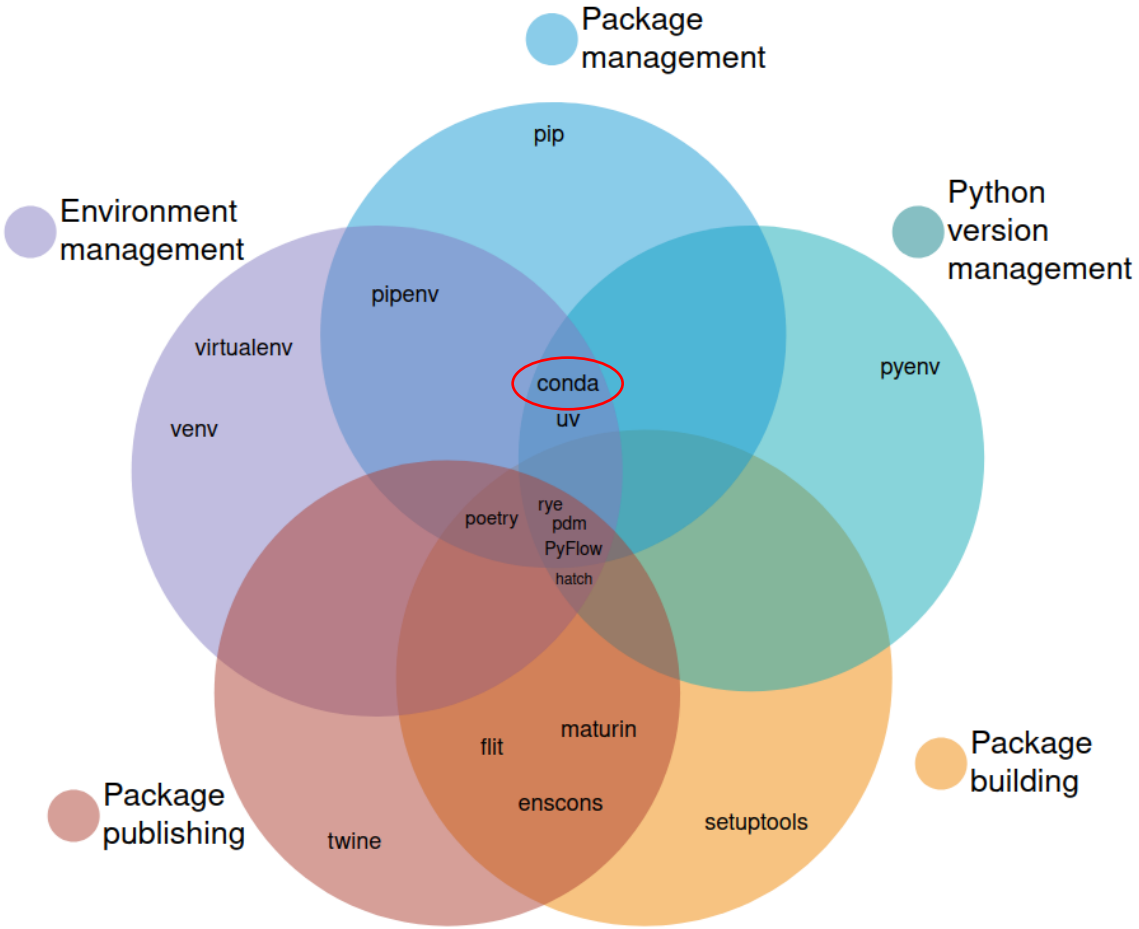
Core Goals

- **Reproducibility**
- **Maintainability**
- **Collaboration**
- **Efficiency**

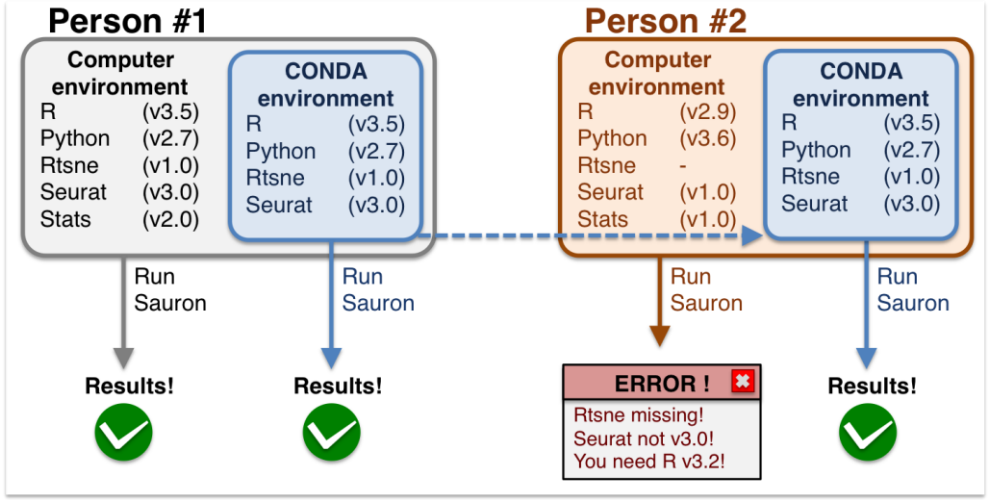
A Sensible Project Structure

```
my-project/
├── .gitignore          # Tells Git what to ignore
├── README.md           # The first thing a visitor will see
├── requirements.txt    # Your project's dependencies
│
├── data/               # All project data
│   ├── raw/            # Original, immutable data. NEVER edit this.
│   └── processed/      # The final, canonical data sets for analysis.
│
├── notebooks/          # Jupyter notebooks for exploration, prototyping, and visualization.
│   └── 01-initial-data-exploration.ipynb
│
├── src/                # Your main source code
│   ├── __init__.py     # Makes src a Python package
│   ├── data_processing.py
│   └── modeling.py
│
├── scripts/            # Any standalone scripts
│   ├── download_data.py
│   └── run_full_pipeline.py
│
└── results/            # Generated output (DO NOT commit to Git)
    ├── figures/        # Figures for your paper/thesis
    └── tables/         # Data tables (e.g., CSVs) summarizing results
```

Managing Your Environment



From **Anna-Lena Popkes**: *An unbiased evaluation of environment management and packaging tools* [1]



From **NBIS** tutorial [2]

[1] https://alpopkes.com/posts/python/packaging_tools/#conda

[2] https://nbisweden.github.io/workshop_omicsint_ISMBECCB/conda_instructions.html

Version Control - Git

Basic commands:

- `git init` : Start tracking a new project
- `git add <file>` : Stage a file for the next "snapshot".
- `git commit -m "Your descriptive message"` : Take the snapshot.

Crucial Rule:

- Commit often and meaningfully, after every logical change.
- Write good commit messages! (not “stuff” or "bug fix." A good message: `Feat: Add function to calculate user engagement`)

Exploiting `.gitignore` File:

- Tell Git what *not* to track.
- Always ignore:
  - Large data files (data/)
  - Virtual environments (.venv/)
  - Secrets and credentials (api_keys.py)
  - Generated output (results/, *.pdf)

Use KU Leuven Gitlab: https://gitlab.kuleuven.be/

Writing Readable Code

The Goal: Code is read far more often than it is written. Make it easy for others (and Future You) to understand.

PEP 8 : Python has an official style guide. Give it a read. Automated solutions exist:

- **Formatter (black):** Automatically reformats your code to be consistent.
- **Linter (flake8):** Checks your code for style errors and potential bugs.

Meaningful Naming: *x* is a bad variable name. *user_id* or *image_height_pixels* is a good variable name.

Docstrings and Comments:

- **Docstrings (for functions/classes):** Explain *what* the function does, its parameters (Args), and what it returns (Returns).
- **Comments (inline):** Explain the *why*, not the *what*.

Writing Readable Code

What does this function do?
What is the difference between *hr* and *hR* arguments?

```
(function) def convert_gaze_3d_nodoc(
    gaze_pitchyaw,
    hr,
    hR: Any | None = None
) -> Any
```

```
(function) def convert_gaze_3d(
    gaze_pitchyaw: Sequence[float] | ndarray,
    hr: ndarray,
    hR: ndarray | None = None
) -> ndarray
```

Converts gaze from head coordinates to camera coordinates.

Args

gaze_pitchyaw
The [pitch, yaw] gaze angles from the model (radians).

hr
The head's rotation vector from solvePnP (shape (3,) or (3,1)).

hR
Optional precomputed head rotation matrix (3x3). If None, it will be computed.

Returns

gaze_vector_cam
The gaze vector in camera coordinate system as a 3-element numpy array.

Is it clearer now?

Writing Readable Code

What is the problem with the functions below?

```
def estimate_head_pose(landmarks_2d, image_size):···

# STEP 2: NORMALIZATION FUNCTIONS

def create_normalized_crop(img, center_3d, hR, cam, focal_norm, distance_norm, roi_size):···

def normalizeEyes(img, face_model_3d, hr, ht, cam):···

def normalizeFace(img, hr, ht, cam):···

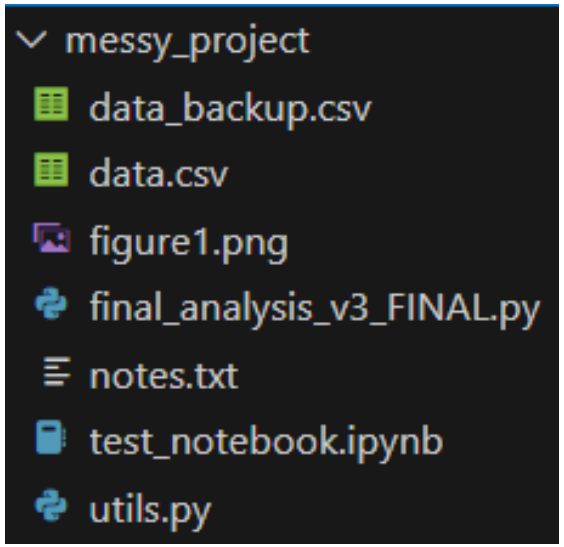
# STEP 3: HELPER FUNCTIONS

def spherical_to_cartesian(pitch, yaw):···

def convert_gaze_3d(gaze_pitchyaw: Union[Sequence[float], np.ndarray], hr: np.ndarray, hR: Optional[np.ndarray] = None) -> np.ndarray:···

def draw_gaze_3d(image, eye_pos_3d_model, gaze_pitchyaw, hr, ht, camera_matrix, length=200.0, color=(0, 0, 255), thickness=2):···
```

Writing Readable Code – Project Organisation



messy_project > final_analysis_v3_FINAL.py > ...

```python
# Author: Alex
# Date: last year sometime
# final version, for real this time

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Step 1: load the data
# I think this is the right file?
raw_data = pd.read_csv('data.csv')

def process_data(df):
    # remove nans and stuff
    df = df.dropna()
    df['new_col'] = df['value'] * np.random.rand() * 100
    return df
```

messy_project > final_analysis_v3_FINAL.py > ...

```python

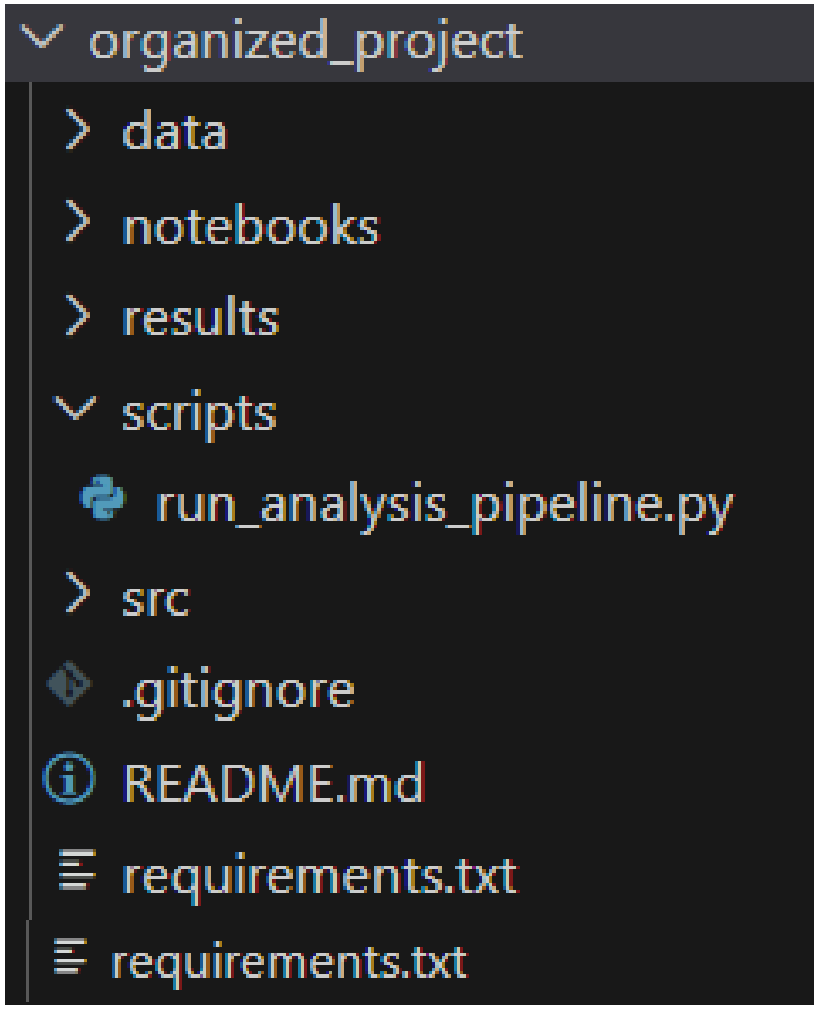
# do processing
processed = process_data(raw_data)
print("Data processed!")

# do some modeling thing
# simple linear regression
model = np.polyfit(processed.index, processed['value'], 1)
print(f"Model coefficients: {model}")

# plot
plt.figure(figsize=(10,6))
plt.plot(processed.index, processed['value'], '.')
plt.title("My Awesome Plot")
plt.savefig('figure1.png') # save the result
print("All done, check figure1.png")
```

Bad organisation, no documentation, random files, no specification => 0/4 for this part of the project's grade

Writing Readable Code – Project Organisation




```python
# This script runs the full analysis pipeline.

import pandas as pd
import matplotlib.pyplot as plt
from src.data_processing import clean_and_enhance_data
from src.modeling import fit_linear_model

RAW_DATA_PATH = 'data/raw/measurements.csv'
PROCESSED_DATA_PATH = 'data/processed/measurements.csv'
FIGURE_PATH = 'results/figures/measurement_trend.png'

print('Running analysis...')
raw_df = pd.read_csv(RAW_DATA_PATH)
processed_df = clean_and_enhance_data(raw_df)
processed_df.to_csv(PROCESSED_DATA_PATH, index=False)
model = fit_linear_model(processed_df.index, processed_df['value'])
print(f'Model fitted with coefficients: {model}')

plt.figure(figsize=(10,6))
plt.plot(processed_df.index, processed_df['value'], '.')
plt.title('Measurement Trend Analysis')
plt.savefig(FIGURE_PATH)
print(f'Pipeline complete. Figure saved to {FIGURE_PATH}')

```

Scripts vs. Notebooks

Jupyter Notebooks (.ipynb) are for...

- Exploration
- Prototyping
- Communication
- Pitfall

Python Scripts (.py) are for...

- Reproducibility
- Reusable Logic
- Automation

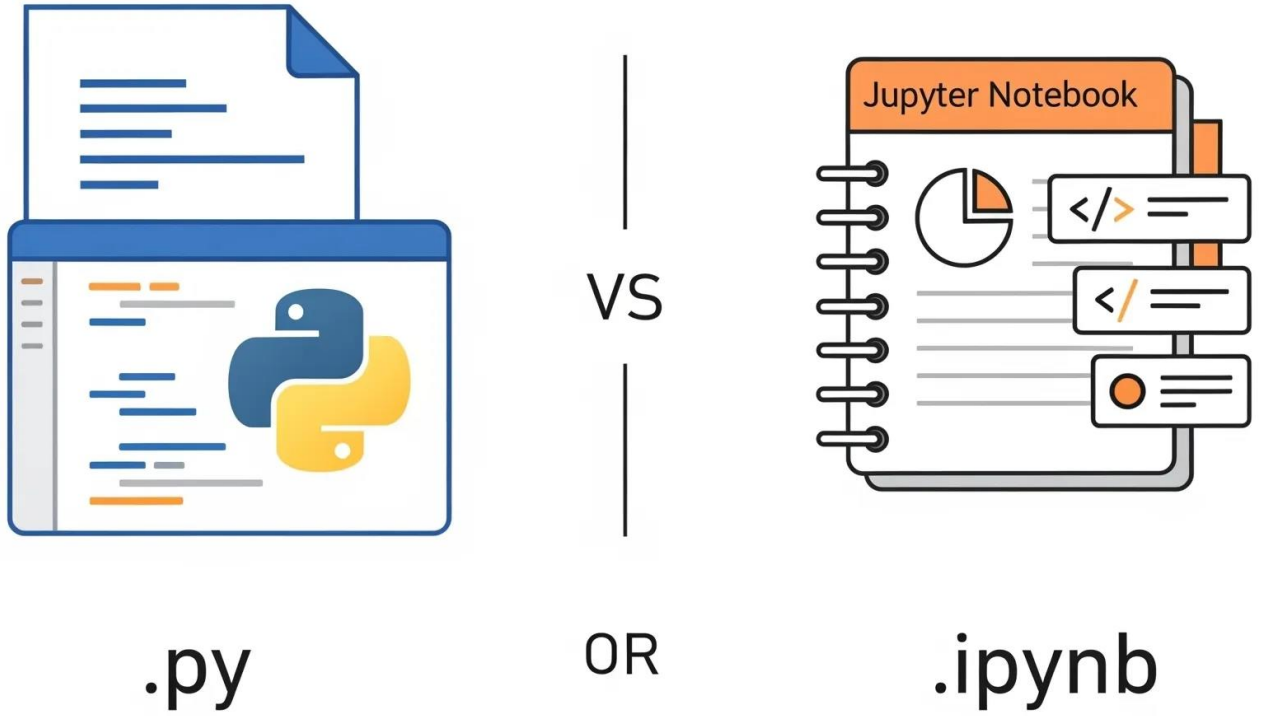


Illustration from Rabail Zaheer article [1]

[1] https://medium.com/@rabailzaheer/what-is-the-difference-between-py-and-ipynb-c880be23a97f

Scripts vs. Notebooks

The Ideal Workflow:

1. Explore and prototype an idea in a notebook in the notebooks/ folder.
2. Once the logic is solid, **refactor** it into a function in a .py file inside src/.
3. Import that function back into your notebook or into a script in the scripts/ folder.
4. Your notebook becomes cleaner, and you now have a reusable, testable function.

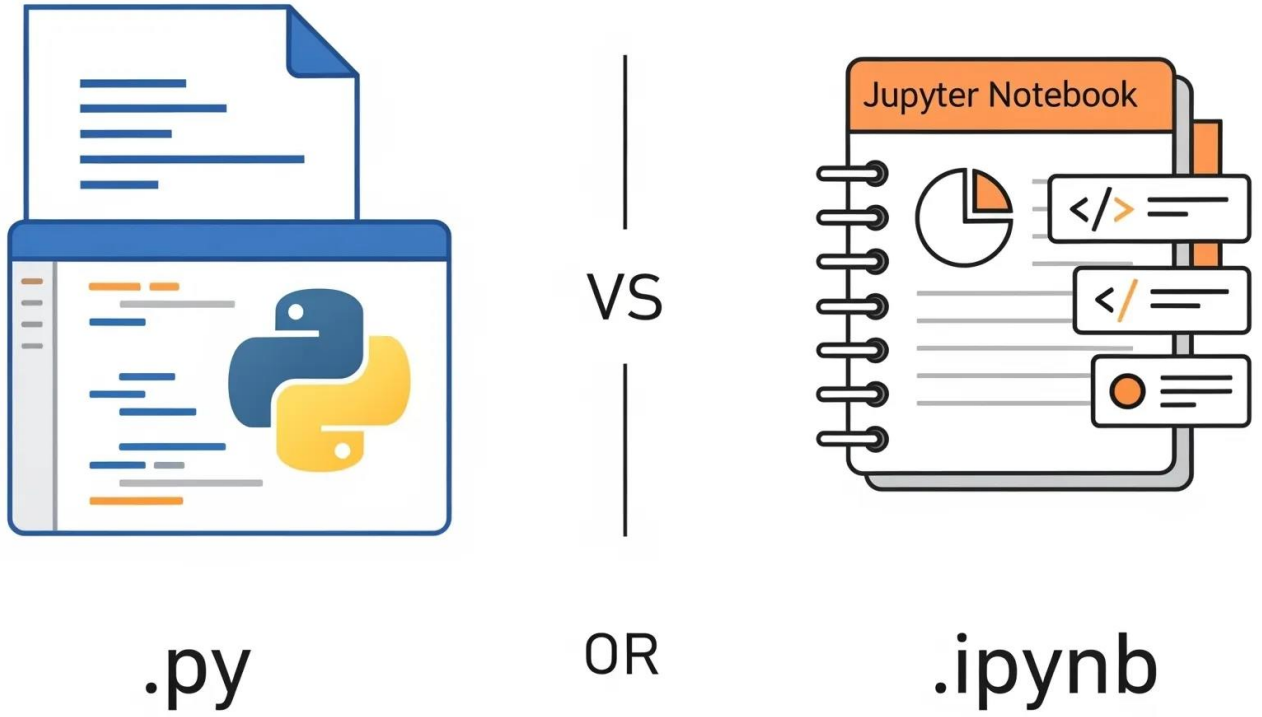


Illustration from Rabail Zaheer article [1]

[1] https://medium.com/@rabailzaheer/what-is-the-difference-between-py-and-ipynb-c880be23a97f

Conclusion

Recap:

1. **Structure:** Use a standard project layout.
2. **Environment:** Isolate dependencies with virtual environments.
3. **Version Control:** Use Git for everything. Commit often, write good messages.
4. **Code Quality:** Use formatters/linters and write clear documentation.
5. **Workflow:** Use notebooks for exploration and scripts for production/pipelines.